

Document history

5.8.3

- Core: Added `mainUI` in the wrapper config. When it's true, NYX integration displays top/bottom bars.

5.8.0

- Core: Added `useCompactNotation`. It formats numbers using compact notation.
- Core: `isSocialCasinoMode` hides `SOC` currency code.

5.7.11

- WindowMessaging: Added `toLobby` event

5.7.9

- LnW Integration (nyx): Removed processing RGS's `RemoteError`.

5.7.8

- Autoplay: Fixed autoplay decreasing counter in the `ScratchApi`.
- Core: Added `jackpotReconnectIntervals` in the wrapper config - exponential backoff strategy.
- UI: Adjusted class names in the top/bottom bar to enable access from jurisdiction config.
- Autoplay: Fixed notification order during autoplay when the reality check is active.

5.7.7

- Autoplay: Fixed `ticket-settled` event and win/loss calculations.

5.7.6

- RGS: Added `updateOgsRgsMap()`, a function that ensure accurate game ID mapping in the `rgsMap` for history mode important to NYX integration.
- RGS: The property `hasGamble` is set to true by default. If the game need to suspend the gamble, it can be done through `gameConfig.gamble.hasGamble` property.

5.7.5

- RGS: Removed rgs verification in multiadapter `getTicket()`..

5.7.4

- RGS: Removed unnecessary game id verification in `getTicket()`.

5.7.3

- AutoplayDialog: Added flag `hideAdvancedAutoplay` in the wrapper config that disables the advanced autoplay dialog
- Cards API: Added `execute()`

5.7.2

- WindowMessaging: Added `freeSpinTrigger` and `freeSpinEnd` events

5.7.1

- AutoplayDialog: Fixed locale language in `formatCurrency()`

5.7.0

- Slot and Cards API: Added `getGame()`
- Cards API: Added `getPreviousTickets()`

5.6.4

- UI: Added `providerLogo` to `BottomBarConfig` for displaying/hiding the provider lobby icon as part of the bottom bar.

5.6.3

- Added support for Card games with `CardApi`.

5.6.0

- Added `enableLangAttribute` parameter to `language-config.json` which allows adding lang attribute to the main html document, enabled by default.
- Described new method `getWinAmount()`.
- iOS: Displaying hide toolbar animaton after splash screen in Safari browser.
- Core: Added versioning with current time parameter on wrapper resources files to avoid caching data

5.5.13

- iOS: swipe-up animation is showing just once
- `fullscreen` can be initilized separately for android and iOS. Swipe/tap animation will be shown by default.

5.5.9

- Created new configuration file `language-config.json`. This dictionary allows you to change the `langPath` path to the directory containing the GameWrapper and `langMap` language files to remap the language URL code to another code.
- Removed `langPath` from Game Wrapper `config`.
- Removed `languageMap.json` configuration file.

5.5.8

- Added `stopConfirmation` in the autoplay configuration

5.5.7

- Added `languageMap` configuration file. The default path is `gamewrapper/languageMap.json`.
5.5.6
- NYX Integration: Multi-game support with unique `config` where `gameId` contains only suffixes
- Displaying swipe animation after splash screen. See `fullscreenAfterLoad` at GameWrapper configuration object.
- Removed `languageMap` from Game Wrapper `config`. ## 5.5.5
- Added `languageMap` to Game Wrapper `config`. ## 5.5.3
- New method `showJurisdiction()` for showing or jurisdiction ui config parameters. ## 5.5.2
- Added `jurisdiction`

5.5.0

- Updated UI Config section with new parameters

5.0.0

- Updated Multi-game support section. Changes are mostly clarifying bonus engines (RGS Bonus Engine and NYX Free Rounds) requirements.
- Various changes through the whole guide because of the move from `clientConfiguration` in RGS config to `gameConfig` in wrapper config.
- Added Bonus Engines support section. Provides info about bonus engines requirements.
- Added `isSocialCasinoMode` to GameWrapper Configuration + info about `currencyDisplay`.

4.0.7

- Clarified the instructions for jackpot notifications usage.

3.2.6

- Added info on payline amounts and denomination values support in `set-stake-limits` event payload and in `betStops` `postMessage` event.

3.2.5

- Slot API: New and updated methods supporting paylines amount:
 - `getAllLines()`
 - `getLineOptions()`
 - `getDefaultLines()`
- Keno API: New methods supporting denomination values:
 - `getAllDenominations()`
 - `getDenominationOptions()`

3.2.3

- New `GameWrapper` config parameter - `enableJackpotNotifications`.

3.1.2

- New autoplay configuration parameter - `stopOnJackpot`.

3.1.1

- Clarified multi game initialization.

3.1.0

- Multi-game support.

3.0.3

- Added game flow diagrams.

3.0.0

- Complete overhaul of the Usage Guide.
- Better overview of wrapper modules.
- Installation guide.
- Usage flow overview.
- Wrapper usage changes - `GameWrapper` should now be used as a proper class.
- Wrapper configuration can be loaded from an external JSON file.
- Wrapper configuration has been extended:
 - It is possible to enable/disable specific UI elements
 - It is possible to disable certain message types

- It is possible to specify the currency data path now.
- New RGS methods:
 - `getJwt()` to get the current JWT,
 - `getAllStakes()` to get a list of stakes supported by the RGS,
 - `getStakeOptions()` to get a reduced list of stakes allowed by both the RGS and the operator,
 - `getDefaultStake()` to get the default stake,
 - `get/setTicketData()` for linking data with a specific ticket,
 - `sendCustomRequest()` to make a custom RGS request
- Slot API and Keno API have been generalized under Simple Flow API.
- Added *gamble* feature.
- Added *progressive jackpots*.
- Documented ‘set-stake-limits’ event, fired when available stakes change.
- New section on *turbo mode*.
- New reasons for autoplay stop:
 - ‘losslimit-required’,
 - ‘single-winlimit-reached’,
 - ‘insufficient-funds’
 - ‘spin-error’
- Added a section on window messaging.

2.3.0

- Updated history replay guidelines.
- `formatCurrency()` API has changed

2.2.0

- Updated Keno free spin API.

2.1.0

- Added Fullscreen section.

2.0.0

- New game type - Keno.
- Keno and Slots have a new method: `setFreeSpin()`.

1.0.0

- Added an overview of fixed odds methods.

0.8.2

- New init parameter – `langPath`.

0.8.1

- Added section **Errors** that links to Error Handling Guidelines.

0.8

- Added notes on blocked requests and GameWrapper states.

0.7

- Game client can now set game name during init or by calling a method.
- Breaking change: splash screen now waits for game wrapper assets to load before hiding.
- `GameWrapper.ui.ready` will inform when the UI is ready and the splash screen is about to close.
- Added notes on currency data and language data.

Overview

GameWrapper is a library which provides a standardized UI and a set of tools for developing games that work with Gromada Games RGS.

The wrapper exposes multiple features to game developers:

- **RGS API** with gamble feature support for fixed odds, slot, keno and cards games,
- **UI** - a set of standardized visual elements with an API to interact with them,
- **Autoplay** mechanism for triggering multiple consecutive purchases,
- **Reality check** handling where required by jurisdiction,
- **Fullscreen management** on mobile devices,
- **Currency formatting**.

Using GameWrapper

Installation

GameWrapper is available at <https://git.fincore.com/gromada-public/gamewrapper.git>. It can be installed as an npm package, cloned from the repository or installed standalone.

NPM install

```
npm install git+https://git.fincore.com/gromada-public/gamewrapper.git
```

The wrapper can then be used as an ES6 module:

```
import GameWrapper from '@gromada/gamewrapper';

const wrapper = new GameWrapper();
```

Standalone install

The wrapper may also be installed by cloning the repository:

```
git clone https://git.fincore.com/gromada-public/gamewrapper.git
```

or downloading the latest package.

The relevant files should then be copied to the game directory or included in the build pipeline.

To use a standalone (IIFE build) it should be included in the game HTML page before the game logic:

```
// minified
<script src="path/to/gamewrapper.min.js"></script>
```

or

```
// debug version
<script src="path/to/gamewrapper.js"></script>
```

Alternatively, the module-based interface is available by importing from *gamewrapper.es.js*.

CSS

For the wrapper UI to work, it is necessary to include the *gamewrapper.css* on your HTML page.

```
<link rel="stylesheet" href="path/to/gamewrapper.css">
```

If your build system supports it, you may also be able to import CSS like this:

```
import '@gromada/gamewrapper/gamewrapper.css';
```

GameWrapper class

The document assumes that the wrapper is being used through an instance of the *GameWrapper* class.

In ES6 builds the wrapper class is the default export of the '@gromada/gamewrapper' module:

```
import GameWrapper from '@gromada/gamewrapper';
```

In legacy (IIFE) builds, the wrapper class is available as a member of the *GameWrapper* namespace:

```
GameWrapper.GameWrapper
```

Note: This is different from wrapper versions before 3.0.0, where the API was available through static methods on the **GameWrapper** namespace.

Legacy API

The legacy API is still available in IIFE builds to allow for backwards-compatibility. Most of the methods are mirrored as static member functions of the **GameWrapper** namespace.

Using this is deprecated and should be avoided in new wrapper integrations.

Usage flow

Initializing

1. Initialize the wrapper.
2. Load game assets. When the wrapper is initialized, update loading progress.
3. Setup game, including wrapper event listeners.
4. Perform the initial RGS call:
 - slots, keno and cards: `refresh()`,
 - fixed odds: `getGame()`.
5. Set available stakes and set the default based on the response.
6. Check game mode.
 - If set to history, handle history replay.
7. Check balance (`getAccount()`).
8. Call `wrapper.gameReady()`.
9. Update stake list if *'set-stake-limits'* event is received.
10. *Fixed odds only:* Check for unsettled ticket.
11. Wait for `ui.ready` to resolve.
12. Display intro animations.

Game flow

After initialization, wrapper-game interaction follows the loop below:

1. Player selects a stake.
 - Inform wrapper of stake change.
2. On button press, issue purchase ticket request.
3. Handle purchase response.
 - Display game animations (spinning, rolling, scoring, etc.), or
 - Handle errors;
4. Execute any game features, if they are supported.
5. Handle feature responses (similar to #3).
6. Display final animations.
7. Close the ticket.

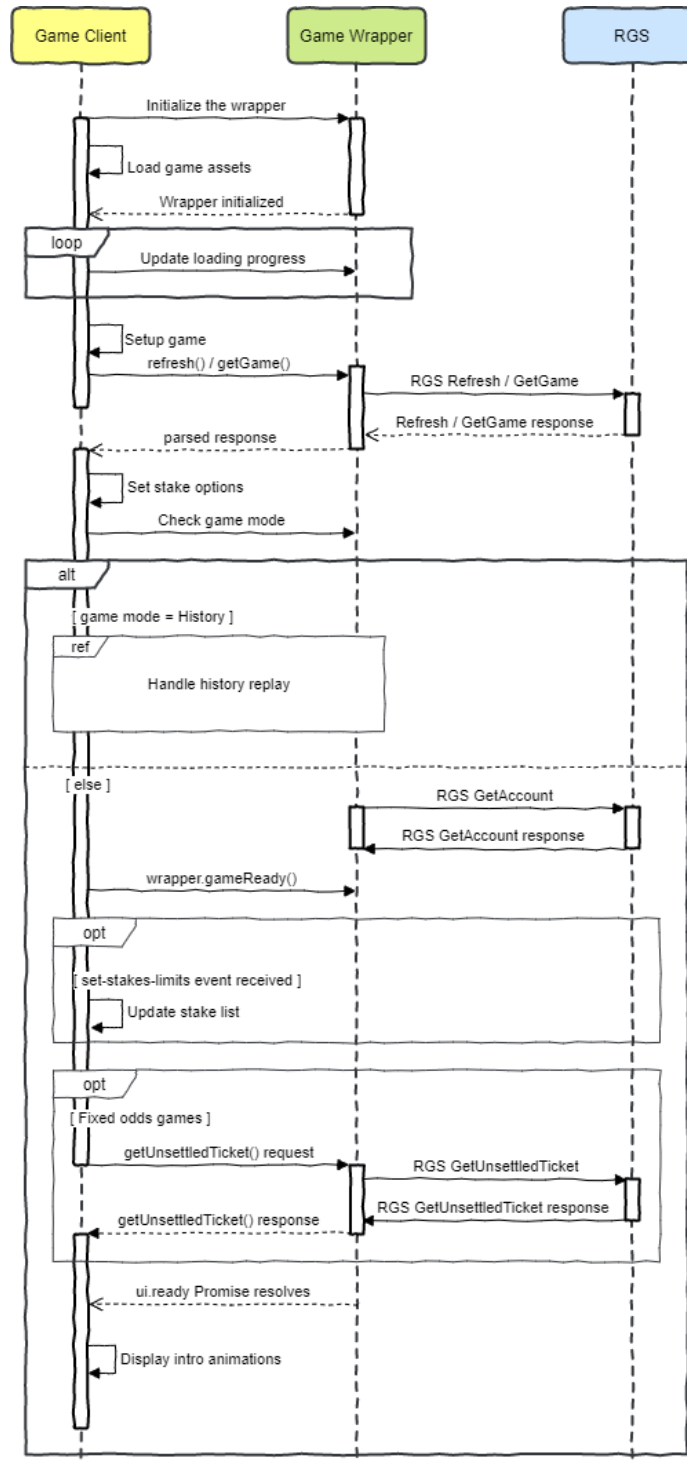


Figure 1: Overview of the initialization flow

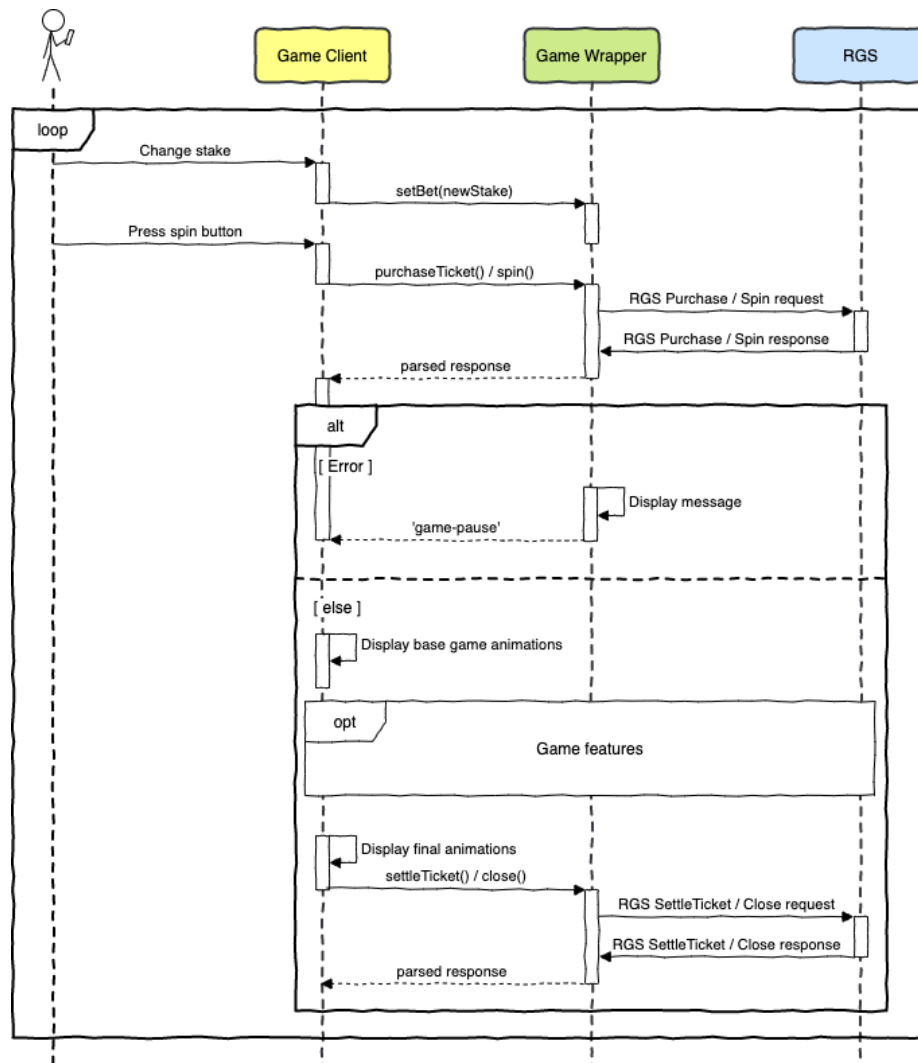


Figure 2: Overview of basic game flow

Unfinished game

At the end of the initialization phase, the game client may receive details of a previous game that hasn't been completed. This is known as an *unsettled*, *broken* or *unfinished* game.

In case there is an unfinished game, the game client should set it up and fast-forward to the appropriate state of the game flow above. This will be one of the steps from #3 to #6.

1. Unsettled game data received (through `refresh()` or `getUnsettledTicket()`).
2. Set up existing game based on data.
3. If there are features to be resolved: execute features (step #4 of game flow).
4. If there are no game actions remaining: display final animations and close the ticket (steps #6 and #7).

History replay

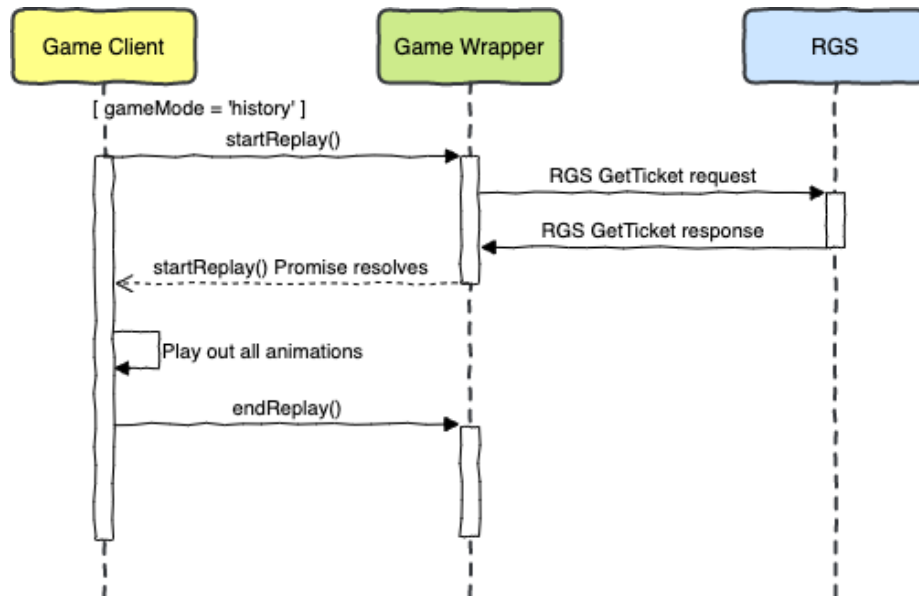


Figure 3: Overview of history replay flow

History replay mode is a feature which allows players and operators to replay any previously played games.

The flow for history replay is slightly different than the regular game flow. The ticket structure may slightly differ too. For example, for slot, keno and cards games, the history replay data will contain the spin and all free spins in a single response.

1. Check game mode.
2. If game mode is *'history'*, call `wrapper.startReplay()` when the game is ready.
3. Parse game response and replay according to game flow.

For detailed information about history replay, please see the appropriate section further in this document.

Initialization

The wrapper is initialized by instantiating a `GameWrapper` instance with configuration parameters. The wrapper will then set up the appropriate RGS API, UI and additional features based on these parameters.

The constructor signature looks like this:

```
GameWrapper(config: GameWrapperConfig | string)
```

where `config` can either be a **GameWrapper configuration object** or a URL that points to a JSON file that contains the GameWrapper Configuration object.

Example:

```
import GameWrapper from '@gromada/gamewrapper';

const wrapper = new GameWrapper({
  gameId: 'golden-rooster',
  gameType: 'slot',

  // Optional parameters below
  gameName: 'Golden Rooster',
  rpcUrl: 'https://rgs.golden-rooster.com',
  content: canvasElement,
  container: canvasElement.parentElement,
});

wrapper.ready.then(() => setupGame);
```

Or alternatively:

```
import GameWrapper from '@gromada/gamewrapper';

const wrapper = new GameWrapper('config.json');
wrapper.ready.then(() => setupGame);
```

When loading configuration from a URL path, it is possible to specify **configuration overrides**. This is useful for the properties that need to be determined at runtime. For example:

```
import GameWrapper from '@gromada/gamewrapper';
```

```
const wrapper = new GameWrapper('config.json', {
  content: game.canvas
});
wrapper.ready.then(() => setupGame);
```

The `GameWrapper` constructor sets up the wrapper asynchronously, as it might have to wait for configuration and other assets. When `GameWrapper.prototype.ready` resolves, the wrapper is ready for use and game setup may continue.

`GameWrapper.prototype.ready` will be activated when `GameWrapper.prototype.initialized`, currency, and language information are resolved. `GameWrapper.prototype.initialized` will be resolved when RGS API, UI and additional features are set up. Currency and language information will be resolved when they are loaded from corresponding paths described in `GameWrapper` configuration object.

It is possible to continue some game setup operations before the wrapper is fully initialized, as long as they don't rely on the wrapper.

GameWrapper configuration object

The configuration object may contain the following properties:

- `gameId` *{string | string[]}* - the game ID or an array of game IDs.
- `gameType` *{string}* - type of the game, determines which RGS engine to use. Can be either 'slot', 'keno', 'cards' or 'fixed-odds'.
- `metaGameId` *{string}* - (optional) meta game ID. Serves as a common identifier for a group of game IDs. Should be used if available on RGS.
- `gameName` *{string}* - (optional) the name of the game (e.g. "Golden Rooster"). Used when displaying the game title in wrapper UI. If not provided, the game name will be set to `gameId`.
- `rpcUrl` *{string}* - (optional) the RPC url to use for communication with RGS. It may be useful to set a custom `rpcUrl` during development, or in case of a customized RGS deployment. The value may be an absolute url or a relative one (relative to the game's host). The default rpc url is set to `<HOSTNAME>/api/rpc`.
- `currencyData` *{string}* - (optional) relative or absolute path to the currency data file.
- `currencyDisplay` *{'symbol' | 'narrowSymbol' | 'code' | 'name'}* - (optional) display style for currency designation in currency labels.
- `isSocialCasinoMode` *{boolean}* - {optional} in social casino mode currency symbol should not be displayed.
- `useCompactNotation` *{boolean}* - {optional} whether to use compact currency notation (e.g. '10K' instead of '10,000').

- **content** *{HTMLElement | string}* - (optional) **HTMLElement** or a selector string for fetching the **HTMLElement** which contains the game content. This element will be embedded within the GameWrapper UI. If **content** isn't set, GameWrapper will initialize without the UI component.
- **container** *{HTMLElement | string}* - (optional) **HTMLElement** or a selector string for fetching the **HTMLElement** which should contain the GameWrapper UI. This value should be set in situations where the GameWrapper UI should not be a direct child of the document body. The default value is the document body.
- **fullscreenContent** *{HTMLElement | string}* - (optional) Root object to be used for the Fullscreen mode. The default value is the document body.
- **langPath** *{string}* - (optional) Relative or absolute path to language files.
- **jurisdictionPath** *{string}* - (optional) Relative or absolute path to language files.
- **ui** *{UIConfig}* - (optional) UI configuration object. It can be used to enable or disable specific UI elements that are provided by the wrapper.
- **messages** *{boolean | MessageDialogConfig}* - (optional) Message display configuration. If the configuration object is supplied, it can specify **either** which messages to include (whitelist mode), or which ones to exclude (blacklist mode). If this parameter is set to false, the message dialog will never display anything. It's important to note that doing this will also disable autoplay messages.
- **autoplay** *{AutoplayWrapperConfig}* - (optional) Autoplay configuration. It includes flags for hiding Advanced Autoplay settings and for making Loss limits mandatory. Note that these are opposing flags, but **hideAdvancedAutoplay** has seniority.
- **postMessage** *{boolean | PostMessageConfig}* - (optional) Some lobbies communicate with the game through **window.postMessage()**. This parameter is used to enable postMessage communication and configure it.
- **fullscreen** *{boolean | {android?: boolean | ios?: boolean}}* - (optional) Whether to enable fullscreen handling. Turned on by default on both android and iOS.
- **fullscreenAfterLoad** *{boolean}* - (optional) Show fullscreen overlay **after** loading. By default, fullscreen overlay is shown after splash screen.
- **forceOrientation** *{'landscape' | 'portrait'}* - (optional) Enforce a certain orientation on mobile devices. If set, the wrapper will show an animation asking the user to rotate the device when the device is in the wrong orientation.
- **enableJackpotNotifications** *{boolean}* - (optional) Whether to enable status notifications for jackpots or not. Disabled by default. These no-

tifications include jackpot pool amount updates and win events. When enabled, notifications are pushed to GameWrapper from RGS.

- **jackpotReconnectIntervals** *{number[]}* - *{optional}* Sequence of increasing delay intervals (in milliseconds) between consecutive reconnection attempts when subscription calls fail. The array represents an exponential backoff strategy.
- **rsiLobbyRestriction** *{boolean}* - *(optional)* Whether checking the lobby URLs from where the game is launched is necessary or not. Unnecessary by default.
- **isCdnSupportEnabled** *{boolean}* - *(optional)* Whether to enable CDN domain prefixes support or not. Enabled by default.
- **enableBonusOffersSupport** *{boolean}* - *(optional)* Whether to enable RGS bonus engine support or not. Disabled by default.
- **mainUI** *{boolean}* - *(optional)* Whether to use main UI top/bottom bar settings, false by default. If true, top/bottom bars will be shown by UIConfig settings.
- **commonUIPosition** *{string}* - *(optional)* Preferred position of CommonUI toggle bar on mobile game interface. Supported values are: 'left' / 'right' / 'centre'. If not supplied or in case of wrong value, 'centre' is used.
- **gcmConfigPrefix** *{string}* - *(optional)* Relative or absolute prefix for GCM config files.
- **showLogo** *{boolean}* - *(optional)* Whether an Aruze logo should be shown during game loading. If value is not specified, logo will be shown by default.
- **isIosPlatform** *{boolean}* - *(optional)* Whether a game is deployed on iOS platform.
- **glUtilPath** *{string}* - *(optional)* Custom path to GLUtil library

UIConfig There are two ways of customizing the wrapper UI. One is to override the wrapper CSS, the other one is to disable elements in the wrapper configuration.

There is an important difference between these two methods. When UI elements are disabled through configuration parameters, they will never be added to the DOM at all.

The UIConfig object has the following structure. All properties are optional and enabled by default:

- **splash** *{boolean / SplashConfig}* - *(optional)* Splash screen configuration. If set to false, splash screen will be disabled.

SplashConfig interface:

- **logo** *{boolean}* - (optional) Enable/disable logo during splash screen.
- **progressBar** *{boolean}* - (optional) Enable/disable progress bar during splash screen.
- **topBar** *{boolean / TopBarConfig}* - (optional) Top bar configuration. If set to false, top bar will be disabled.

TopBarConfig interface:

- **btnLobby** *{boolean}* - (optional) Enable/disable lobby button in top bar.
- **btnSound** *{boolean}* - (optional) Enable/disable sound button in top bar.
- **title** *{boolean}* - (optional) Enable/disable game title display in top bar.
- **date** *{boolean}* - (optional) Enable/disable date in top bar.
- **clock** *{boolean}* - (optional) Enable/disable clock in top bar.
- **elapsedTime** *{boolean}* - (optional) Enable/disable elapsed time stopwatch in top bar.
- **btnMenu** *{boolean}* - (optional) Enable/disable menu button in top bar.
- **bottomBar** *{boolean / BottomBarConfig}* - (optional) Bottom bar configuration. If set to false, bottom bar will be disabled.

BottomBarConfig interface:

- **btnMenu** *{boolean}* - (optional) Enable/disable menu button in bottom bar.
- **btnLobby** *{boolean}* - (optional) Enable/disable lobby button in bottom bar.
- **btnHelp** *{boolean}* - (optional) Enable/disable help button in bottom bar.
- **btnSound** *{boolean}* - (optional) Enable/disable sound button in bottom bar.
- **btnAutoplay** *{boolean}* - (optional) Enable/disable autoplay button in bottom bar.
- **lblBalance** *{boolean}* - (optional) Enable/disable balance label in bottom bar.
- **lblBonusRounds** *{boolean}* - (optional) Enable/disable bonus rounds in bottom bar.

- **lblBet** *{boolean}* - (*optional*) Enable/disable stake label in bottom bar.
- **lblWinAmount** *{boolean}* - (*optional*) Enable/disable win amount label in bottom bar.
- **lblNetAmount** *{boolean}* - (*optional*) Enable/disable net amount label in bottom bar.
- **lblGameMode** *{boolean}* - (*optional*) Enable/disable game mode label in bottom bar.
- **elapsedTime** *{boolean}* - (*optional*) Enable/disable elapsed time stopwatch in bottom bar.
- **providerLogo** *{boolean}* - (*optional*) Enable/disable provider logo in bottom bar.
- **help** *{boolean}* - (*optional*) Enable/disable Game Wrapper’s help screen.
- **autoplay** *{boolean}* - (*optional*) Enable/disable Game Wrapper’s autoplay menu

MessageDialogConfig Message dialog configuration can either function as a blacklist or as a whitelist of message types that can be shown. Because of this **include** and **exclude** properties are mutually exclusive and only one should be specified.

A detailed list of message types is available in the message dialog section.

- **exclude** *{string[]}* - (*optional*) A list of message types which to ignore. Message dialog won’t show for these messages.
- **include** *{string[]}* - (*optional*) A list of message types which to allow. Message dialog won’t show for any other message type.
- **showLobbyButton** *{boolean}* - (*optional*) If set to false, message dialog will never show buttons that take player to the lobby. Required for some operators.

AutoplayWrapperConfig Autoplay configuration currently has next options:

- **hideAdvancedAutoplay** *{boolean}* - (*optional*) If set to true, advanced button and dialog won’t be shown.
- **requireLossLimit** *{boolean}* - (*optional*) If set to true, autoplay won’t start until a loss limit is set.
- **stopOnJackpot** *{boolean}* - (*optional*) Enable or disable the *stop on jackpot* switch in the autoplay dialog. By default, the switch is hidden and disabled.
- **stopConfirmation** *{boolean}* - (*optional*) Enable or disable displaying a message when autoplay is stopping. By default, the message will be shown.

Game modes

Once the wrapper is initialized, an appropriate game mode will be set in `GameWrapper.prototype.gameMode`. Possible values are:

- `realmoney` - for real money play mode,
- `demo` - for demo / play for fun mode,
- `history` - for history replay.

RGS API

The RGS API is available through `GameWrapper.prototype.rgs`.

The exact interface and the available methods depend on the `gameType` set during initialization. Currently available game types are:

- Fixed odds
- Slots
- Keno
- Cards

Slot, Keno and Cards interface are based on the same Simple Flow API.

All RGS methods return a **Promise** which resolves to a parsed RGS response object. Full details about the methods and their responses are specified in separate documents.

Note: To avoid blocked requests, it is recommended to check `wrapper.state` before calling RGS methods and only call them if the state is **active**. If the state is **paused**, the client should wait for a **game-resume** event before calling the desired method.

Common methods

Methods that communicate with RGS

- `getAccount()`
- `getTicketData(ticketId: string)`
- `setTicketData(data: string | object, ticketId?: string)`
- `sendCustomRequest(service: string, method: string, args: any[])`

Helper methods

- `getJwt()`
- `getAllStakes()`
- `getStakeOptions()`
- `getDefaultStake()`

Promise

- `configured`

getAccount() Get user's current balance data including the currency. Calling this method will trigger a balance update throughout the whole GameWrapper.

In order to minimize server requests, **getAccount()** should be called only after an expected balance change (e.g. on game start, after spin and after close).

Example

```
wrapper.rgs.getAccount()
  .then(function (data) {
    console.log('Current balance: ' + data.balance + ' ' + data.currency)
  }).catch(handleErrors);
```

getTicketData(ticketId?: string) Get data that was associated with a certain ticket.

If **ticketId** isn't specified, the wrapper will get data associated with the current ticket.

Example

```
wrapper.rgs.getTicketData('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then(function (data) {
    console.log('Ticket data: ' + data);
  }).catch(handleErrors);
```

setTicketData(data: string | object, ticketId?: string) Associate data with a ticket.

data may either be a JSON string or a serializable javascript object. The object will be stringified before sending the request to RGS.

If **ticketId** isn't set it will default to the current ticket ID.

Example

```
wrapper.rgs.setTicketData({ color: 'red' })
  .then(function (data) {
    console.log('Ticket data: ' + data);
  }).catch(handleErrors);
```

sendCustomRequest(service: string, method: string, args: any[])

Send a custom RGS request through the wrapper. This is useful when the client wants to use a feature which is supported by the RGS, but not implemented by the wrapper.

Should be used with caution. Best course of action is to ask first if this is the right way to implement required functionality. That is to avoid unnecessary debugging and delays.

- `service` is the RGS service name
- `method` is the RGS method name
- `args` is an array of arguments for the method. May be an empty array for no arguments.

Example

```
const gameId = /* some gameId related logic */;
wrapper.rgs.sendCustomRequest('SlotService', 'Refresh', [gameId])
  .then((data) => console.log('Refresh response: ', data))
  .catch(handleErrors);
```

getJwt() Get the latest JWT.

Returns a string.

Example

```
let jwt = wrapper.rgs.getJwt();
console.log('Current JWT: ' + jwt);
```

getAllStakes() Get a list of all stakes supported by the RGS (as defined by the `gameConfig` section of wrapper configuration).

Returns an array of numbers.

Example

```
let stakes = wrapper.rgs.getAllStakes();
console.log('Supported stakes: ' + stakes.join(', '));
```

getStakeOptions() Get a list of stakes that the game is allowed to use. This includes any limitations set by the operator.

The difference between `getAllStakes()` and `getStakeOptions()` is that `getStakeOptions()` includes additional limitations which are set by the game operators. The result of this method is thus always a subset of `getAllStakes()`.

For the purpose of presenting a choice of stakes, this is the method that should be used.

Returns an array of numbers.

Example

```
let stakes = wrapper.rgs.getStakeOptions();
console.log('Available stakes: ' + stakes.join(', '));
```

getDefaultStake() Get the default stake (as defined by the `gameConfig` section of wrapper configuration).

Returns a number.

Example

```
let defaultStake = wrapper.rgs.getDefaultStake();
console.log('Default stake: ' + defaultStake);
```

configured promise An RgsAdapter instance exposes a **configured** promise which resolves when RgsAdapter has been fully configured.

In fixed odds games, this happens after `rgs.getGame()`, while in simple flow games this happens after the first `rgs.refresh()` or `rgs.getTicket()`.

Example

```
wrapper.rgs.configured.then(function () {
  let stakes = wrapper.rgs.getStakeOptions();
  console.log('Available stakes are: ' + stakes.join(', '));
});
```

Fixed odds

For fixed odds games (`gameType: 'fixed-odds'`) the available methods are:

- `getGame()`
- `getUnsettledTicket()`
- `purchaseTicket(bet: number)`
- `settleTicket()`
- `getTicket(ticketId: string)`

getGame() Get game data including stake, prize level and configuration data. `getGame()` should be the first RGS API call after `GameWrapper` is initialized.

Example

```
wrapper.rgs.getGame()
  .then(function (data) { console.log('Available stakes: ' + data.ticketPrice) })
  .catch(handleErrors);
```

getUnsettledTicket() Get an unsettled ticket if it exists. An unsettled ticket may exist if the previous game hasn't been settled properly (e.g. if the user closed the game early, or the game crashed).

It is recommended to call this method before calling `purchaseTicket()`, but it is not required. The wrapper will execute `getUnsettledTicket()` automatically if purchase fails due to an unsettled ticket.

Example

```
wrapper.rgs.getUnsettledTicket()
  .then(function (ticket) {
    if(ticket)
      console.log('Unsettled ticket found. Details: '+JSON.stringify(ticket))
  }).catch(handleErrors);
```

purchaseTicket(bet: number) Purchase a new ticket. In case of a successful purchase, the response will contain ticket data including the selected scenario.

Example

```
wrapper.rgs.purchaseTicket(0.5)
  .then(function (ticket) { console.log('Selected scenario: '+ticket.scenario) })
  .catch(handleErrors);
```

settleTicket() Settle the current ticket. **settleTicket** should be called at the end of each game, before another purchase is made.

Example

```
wrapper.rgs.settleTicket()
  .then(function (data) { console.log('Ticket settled') })
  .catch(handleErrors)
```

getTicket(ticketId: string) Get ticket data for a specific settled ticket. This is usually handled by the GameWrapper during history replay, so it is rare to call this method directly. May be useful for getting historical ticket data. For more details about history replay, see the appropriate section below.

Example

```
wrapper.rgs.getTicket('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then(function (ticket) { console.log('Ticket has scenario: '+ticket.scenario)})
  .catch(handleErrors)
```

Simple Flow API

Simple Flow API is the basis for slot, keno and cards games interface. It is also the base for future game implementations.

The methods for simple flow games are mostly the same, though the RGS responses may differ. The exact responses are documented in additional API specific documents.

The basic idea behind simple flow is that gameplay is split into four consecutive phases: *refresh*, *spin*, *features* (such as gamble and free spin), *close*.

The wrapper keeps track of the next pending action and will display a warning in the console if the game tries to execute an out-of-order action (e.g. if the next action is `close`, but the game tries to call `spin`).

The available methods in simple flow based games are:

- `refresh()`
- `spin()`
- `setFreeSpin(freeSpinType: string)`
- `freeSpin()`
- `close()`

Arguments to `spin()` and `freeSpin()` methods depend on the game type. Other methods are the same for all games.

refresh() Get new game data and the next pending action (*spin*, *freeSpin* or *close*). `refresh()` should be the first RGS API call after `GameWrapper` is initialized.

Example

```
wrapper.rgs.refresh()
  .then(function (data) { console.log('Received game data:', data.game) })
  .catch(handleErrors);
```

setFreeSpin(freeSpinType: string) Set the type of the next free spin game. Required in some games.

- `freeSpinType` - the desired free spin type. RGS checks the requested type and returns an error if the type isn't valid for the current game.

Example

```
wrapper.rgs.setFreeSpin('Y')
  .then(function (data) { console.log('Free spin set. Call freeSpin() now') })
  .catch(handleErrors);
```

freeSpin() Perform a free spin.

Example

```
wrapper.rgs.freeSpin()
  .then(function (data) { console.log('Free Spin done. Next action is ' + data.game.nextAction) })
  .catch(handleErrors);
```

close() Finish the current game and close the ticket.

Example

```
wrapper.rgs.close()
  .then(function (data) { console.log('Ticket closed') })
  .catch(handleErrors);
```

Slots

Methods specific to slot games (`gameType: 'slot'`) are:

- `spin(lineCount: number, gameType: string, betPerLine: number)`
- `getAllLines()`
- `getLineOptions()`
- `getDefaultLines()`

`spin(lineCount: number, gameType: string, betPerLine: number)`

Purchase and start a new spin.

- `lineCount` - number of selected paylines
- `gameType` - one of the game types provided by the game engine. E.g. 'S1D1', 'S2D1', etc.
- `bet` - bet amount for this spin.

Example

```
wrapper.rgs.spin(25, 'S1D1', 1)
  .then(function (data) {
    console.log('Spin done. Next action is ' + data.game.nextAction);
  }).catch(handleErrors);
```

`getAllLines()` Get the list of all payline amounts allowed by the `gameConfig` section of wrapper configuration. All allowed amounts must be actually supported by the RGS.

Returns an array of numbers.

Example

```
let allLines = wrapper.rgs.getAllLines();
console.log('Supported payline amounts: ' + allLines.join(', '));
```

`getLineOptions()` Get the list of payline amounts that the game is allowed to use. This includes any limitations set by the operator.

The difference between `getAllLines()` and `getLineOptions()` is that `getLineOptions()` includes additional limitations which are set by the game operators. The result of this method is thus always a subset of `getAllLines()`.

For the purpose of presenting a choice of payline amounts, this is the method that should be used.

Returns an array of numbers.

Example

```
let lineOptions = wrapper.rgs.getLineOptions();
console.log('Available payline amounts: ' + lineOptions.join(', '));
```

getDefaultLines() Get the default payline amount as defined by the gameConfig section of wrapper configuration.

Returns a number.

Example

```
let defaultLines = wrapper.rgs.getDefaultLines();
console.log('Default payline amount: ' + defaultLines);
```

Keno

These are the methods specific to keno games (gameType: 'keno'):

- spin(bet: number, selectedNumbers: number[])
- freeSpin(selectedNumbers: number[])
- getAllDenominations()
- getDenominationOptions()

spin(bet, selectedNumbers) Purchase and start a new spin.

- bet - bet amount
- selectedNumbers - an array of keno numbers selected by the player

Example

```
wrapper.rgs.spin(1, [5, 13, 25, 26, 33, 54, 62, 68, 69, 71])
  .then(function (data) {
    console.log('Spin done. Next action is ' + data.game.nextAction);
  }).catch(handleErrors);
```

freeSpin(selectedNumbers) Perform a free spin.

- selectedNumbers - an array of keno numbers selected by the player. This may be different selection from the one in the initial spin, but it must be at the same level (i.e. have the same amount of values).

Example

```
wrapper.rgs.freeSpin([5, 13, 25, 26, 33, 54, 62, 68, 69, 71])
  .then(function (data) {
    console.log('Free Spin done. Next action is ' + data.game.nextAction);
  }).catch(handleErrors);
```

getAllDenominations() Get the list of all denomination values allowed by the `gameConfig` section of wrapper configuration. All allowed values must be actually supported by the RGS.

Returns an array of numbers.

Example

```
let allDenominations = wrapper.rgs.getAllDenominations();
console.log('Supported denomination values: ' + allDenominations.join(', '));
```

getDenominationOptions() Get the list of denomination values that the game is allowed to use. This includes any limitations set by the operator.

The difference between `getAllDenominations()` and `getDenominationOptions()` is that `getDenominationOptions()` includes additional limitations which are set by the game operators. The result of this method is thus always a subset of `getAllDenominations()`.

For the purpose of presenting a choice of denomination values, this is the method that should be used.

Returns an array of numbers.

Example

```
let denominationOptions = wrapper.rgs.getDenominationOptions();
console.log('Available denomination values: ' + denominationOptions.join(', '));
```

Cards

This is the method specific to card games (`gameType: 'cards'`):

- `spin(bet: number, selectedCards: number[] [])`
- `freeSpin(selectedCards: number[] [])`
- `execute(commands: string[])`
- `getPreviousTickets`

spin(bet, selectedCards) Purchase and start a new spin.

- `bet` - bet amount

- **selectedCards** - should be a one-dimensional array in the case of single-hand games, where array represents all cards on the screen, where 1 indicates a selected card and 0 stands for a non-selected card

Example

```
wrapper.rgs.spin(1, [[1, 0, 0, 0, 0]])
  .then((data) => {
    // process data received from RGS
  })
```

freeSpin(selectedCards) Perform a free spin.

- **selectedCards** - should be a one-dimensional array in the case of single-hand games, where array represents all cards on the screen, where 1 indicates a selected card and 0 stands for a non-selected card

Example

```
wrapper.rgs.freeSpin([[1, 0, 0, 0, 0]])
  .then((data) => {
    // process data received from RGS
  })
```

execute(commands) Execute commands from the argument's list.

- **commands** - array of commands that should be executed

Example

```
wrapper.rgs.execute(["piqum-bonus:CancelBonus"])
  .then((data) => {
    // process data received from RGS
  })
```

getPreviousTickets() Return the last 10 tickets of the specified game within a 24-hour interval. The elements of the array in the response are formatted same as in GetTicket response.

Example

```
wrapper.rgs.getPreviousTickets()
  .then((data) => {
    // process data received from RGS
  })
```

Gamble

Gamble is a special feature that allows a player to wager their winnings at the end of the game, with the possibility of increasing their gains. This feature is available to all game types as long as the RGS is configured to support it.

It is possible to check if the gamble feature is supported by inspecting the `hasGamble` flag, it's set to `true` by default.

The wrapper exposes following methods for the gamble feature:

- `getGambleOptions()`
- `getGamble(ticketId?: string)`
- `gamble(symbol: string, ticketId?: string)`

hasGamble flag The gamble feature is enabled by default for the game, but it can be disabled by `gameConfig.gamble.hasGamble` property.

getGambleOptions() Get a list of gamble options and their configuration.

For slots and keno games it will return a valid list only if gamble options are defined in the `gameConfig` section of wrapper configuration. If `getGambleOptions()` cannot return a valid list, it will return `undefined`.

For fixed-odds games it's available only after the game setup response is received (see `configured` promise).

Example

```
let gambleOptions = wrapper.rgs.getGambleOptions();
console.log('Known gamble options: ' + gambleOptions.join(', '));
```

getGamble(ticketId?: string) Get gamble data related to a single ticket.

This data includes:

- Whether a gamble is available,
- A reason, if it isn't,
- History of previously played gambles on this ticket.

Argument:

- `ticketId` - (*optional*) the ticket ID for which to check gamble data. If not set `getGamble()` will get the gamble data for the current ticket.

Example

```
wrapper.rgs.getGamble()
  .then(function (gamble) { console.log('Is a gamble available? ' + gamble.isAvailable) })
  .catch(handleErrors);
```

gamble(symbol: string, ticketId?: string) Gamble the player's winnings on a chosen symbol.

- **symbol** - the selected symbol. Must be actually supported by the RGS.
- **ticketId** - (*optional*) the ticket ID for which to check gamble data. If not set **getGamble()** will get the gamble data for the current ticket.

Example

```
wrapper.rgs.gamble('b')
  .then(function (gamble) {
    let lastGamble = gamble.history[gamble.history.length - 1];
    console.log('Gamble win amount: ' + lastGamble.wA)
  }).catch(handleErrors);
```

Progressive jackpots

Progressive jackpots are a special feature that's supported by some game engines. It is a monetary prize which increases each time a game is played but the jackpot is not won. The increase comes from contributions from every game played as a percentage of the stake. When the jackpot is won it automatically gets reset to a predefined initial value.

The wrapper exposes following methods for the progressive jackpot:

- **getJackpots()**
- **getAllJackpots()**
- **getJackpotWins(jackpotId: string, numberOfWins: number)**

The wrapper provides following options for the progressive jackpot:

- Core: **enableJackpotNotifications**
- Core: **jackpotReconnectIntervals**

getJackpots() Returns all active jackpots.

This data includes:

- **id** - Jackpot ID.
- **a** - amount; Current jackpot value.

Example

```
wrapper.rgs.getJackpots()
  .then(function (data) {
    let lastJackpot = data[data.length - 1];
    console.log('Last jackpot ID: ' + lastJackpot.id);
  }).catch(handleErrors);
```

getAllJackpots() Returns all active jackpots. It does not take any arguments.

This data includes:

- **id** - Jackpot ID.
- **is** - **isSuspended**; if true, jackpot is suspended and no contributions or wins are allowed.
- **a** - amount; current jackpot value.

Example

```
wrapper.rgs.getAllJackpots()
  .then(function (data) {
    let lastJackpot = data[data.length - 1];
    console.log('Last jackpot value: ' + lastJackpot.a);
  }).catch(handleErrors);
```

getJackpotWins(jackpotId: string, numberOfWins: number) Returns specified number of winning jackpots in timestamp ascending order.

- **jackpotId** - Jackpot ID. Must be actually supported by the RGS.
- **numberOfWins** - Number of wins. Up to 50.

This data includes:

- **id** - Jackpot ID.
- **a** - Jackpot winning amount.
- **ts** - Win timestamp, UTC.

Example

```
wrapper.rgs.getJackpotWins('wgr_grand_v1', 25)
  .then(function (data) {
    let lastJackpot = data[data.length - 1];
    console.log('Last win timestamp: ' + lastJackpot.ts);
  }).catch(handleErrors);
```

enableJackpotNotifications option Whether to enable status notifications for jackpots or not. Disabled by default. These notifications include jackpot pool amount updates and win events (by all players including current player). When enabled, notifications are pushed to GameWrapper from RGS.

Upon successful subscription, 2 events would start to dispatch from the **rgs** member of GameWrapper.

In case of connection failure, wrapper would continuously try to reestablish connection and resume notifications reception.

These events include:

- **jackpots-pool-message** - Event containing data array with information about all jackpots enabled for current game.
- **jackpots-win-message** - Event containing data array with information about all jackpots that were won since previous win message was received. This event is dispatched only if there were any jackpot wins.

Data items in these arrays have the following fields:

- **id** - Jackpot ID.
- **a** - Jackpot winning amount.

Example

```

wrapper.rgs.on('jackpots-pool-message', ({ detail }) => {
  const jackpotsPoolData = detail;
  for (const jackpot of jackpotsPoolData) {
    console.log(`current amount for jackpot '${jackpot.id}' is '${jackpot.a}'`);
  }
});

wrapper.rgs.on('jackpots-win-message', ({ detail }) => {
  const jackpotsWinData = detail;
  for (const jackpot of jackpotsWinData) {
    console.log(`someone just won jackpot '${jackpot.id}' with amount of '${jackpot.a}'`);
  }
});

```

jackpotReconnectIntervals option Sequence of increasing delay intervals (in milliseconds) between consecutive reconnection attempts when subscription calls fail. The array represents an exponential backoff strategy.

Default values are [1000, 1000, 5000, 10000, 20000] which are equal to [1s, 1s, 5s, 10s, 20s].

After exhausting all intervals, the client will emit a fatal error.

Blocked requests

The wrapper will block all RGS requests when it is in **paused** or **error** state. This happens when a reality check occurs, when a *wallet message* arrives, or when there is an error.

When a request is blocked it will be rejected with the reason **requestBlocked**. The game should then wait for the wrapper to unblock requests, before making any RGS calls.

If the wrapper is in **paused** state, it may unblock requests at a later time, depending on player input. When it does, it will fire a **game-resume** event.

If the wrapper is in **error** state, the game cannot continue, and it will never unblock requests.

Bonus Engines support

Bonus engines are external systems that allow casino or game provider to promote the game through distribution of promotional (bonus) game rounds.

Two bonus engines are currently supported: RGS Bonus Engine and NYX Free Rounds. Both are functioning only in their respective wrapper versions. RGS Bonus Engine in “main” and NYX Free Rounds in “nyx”.

It’s important to distinguish bonus rounds and free spins. While free spins are an in-game feature that are awarded based on the results produced by the game engine itself, bonus rounds are assigned by operators as a promotional tool and are unrelated to the game engine mechanics.

In order to properly support requirements of bonus engines, game client must support these features:

- Listen for *‘stake-change’* events from **wrapper**. Set currently selected stake based on the received value from such event.
- Listen for *‘set-stake-limits’* events from **wrapper**. Set stakes, payline amounts, denominations and subgames available to the player once such event is received.
 - If current selection is not among the allowed options, then current selection must be set to one of the allowed ones.
 - If only one option is allowed in any category, then it’s selection controls must be disabled. Also, the opposite case must be supported as well.
- Listen for *‘set-enabled’* events from **wrapper.autoplay**. Enable / disable autoplay UI controls based on the received value from such event.

Multi-game support

A single game client may use multiple game configurations on the RGS. This is usually done to allow the game to switch between different sets of outcomes, but could, in theory, be used to present multiple games to the player.

Parent game can be called “metagame”, “multi-game” or “multi-id game” depending on the context. Child games are called “subgame”.

Setting up To set up multi-game support, it is necessary to specify an array of subgame IDs in the wrapper configuration object. It is also highly recommended to set **metaGameId** if it is available.

If **metaGameId** is set, the wrapper can optimize RGS requests and perform less network requests. This leads to less waiting and a better user experience.

Example configuration:


```

{
  "metaGameId": "extreme-dragon",
  "gameId": ["extreme-dragon-el8", "extreme-dragon-el18", "extreme-dragon-el38", "extreme-d
  "gameType": "slot",

  // rest of the configuration
}

```

Multi-game interface The RGS interface is slightly different when working with multiple games.

The main `rgs` member now represents a multi-game adapter. It can be used to perform actions which are common to all games.

It also contains a new method `game(gameId: string)` which returns a subgame specific RGS API instance.

When using multi-game support, it is **important** to follow these guidelines:

- Execute the game initialization call (`initGame()`) on the common `rgs` interface. `initGame()` should be used instead of `refresh()` and `getGame()` / `getUnsettledTicket()`. This makes sure that all the games are updated and checked for any unsettled tickets.
- Execute game flow methods (purchase, game features and settle) through their game-specific adapters (`rgs.game(gameId)`).
- Execute any other methods on the common `rgs` interface.

Examples

```

// get account
rgs.getAccount();

// init games
rgs.initGame();

// purchase ticket for 'extreme-dragon-el68'
rgs.game('extreme-dragon-el68').spin(253, 'EL68', 1);

// free spin
rgs.game('extreme-dragon-el68').freeSpin();

// gamble
rgs.gamble('h');

// settle the game
rgs.game('extreme-dragon-el68').close();

```

```
// get ticket data
rgs.getTicketData(ticketId);
```

NYX multi-game support *Game Wrapper* via RGS gets `rgsGameId`. It is required for performing actions such as *refresh/spin/freespin/close* etc. It means that *Game Wrapper* already has the root name for sub-games and `config` just needs a suffixes for games. Also, fields in `config` must not contain prefixes for operators such as *gn-/rt-/mgm-*,

```
//not correct
"gameId": ["gn-extreme-dragon-el8", "gn-extreme-dragon-el18", "gn-extreme-dragon-el38", "gn-
//correct
"gameId": ["-el8", "-el18", "-el38", "-el68", "-el88"]
```

```
//not correct
"bonusRoundsGameId": "gn-extreme-dragon-el8"
//correct
"bonusRoundsGameId": "-el8"
```

```
//not correct
"gcmConfigPrefix": "/gn-games/extreme-dragon/",
//correct
"gcmConfigPrefix": "/games/extreme-dragon/",
```

Bet Multipliers The wrapper currently requires `betMode` and `betMultipliers` in the game configuration. For now only one bet mode is supported - ‘single-multiplier’.

In ‘single-multiplier’ mode, `betMultipliers` must contain a record named ‘single’ with only one number. This number would be used as a bet multiplier for internal functions. `betMode`

For a game which uses a bet multiplier of 60, this should be added to `gameConfig`:

```
"betMode": "single-multiplier",
"betMultipliers": {
  "single": [60]
}
```

For non-multi-game configurations, these two parameters are used only by:

```
SlotApi.getCurrentMultiplier()
SlotApi.getDefaultStake()
```

Some games might have game modes with different bet multipliers. Since there is no way to specify more than one multiplier, the methods above will almost always return incorrect values.

Workaround

Games with multiple multipliers can avoid multiplier issues by setting the bet multiplier to 1, and then doing the correct multiplier calculations on the client side. I.e. use:

```
"betMode": "single-multiplier",
"betMultipliers": {
  "single": [1]
}
```

And then multiply the result of `getDefaultStake()` with the correct multiplier in-game.

Bonus rounds If a casino or game provider supports promos granting a player bonus rounds, following convention must be followed: a player can receive bonus rounds only for one subgame of a multi-ID game. To have some flexibility, operators can provide bonus rounds with different stake options for that subgame. All stake options have to be allowed for said subgame on the RGS.

Game configuration for bonus rounds It is necessary to specify ID of the subgame that can receive bonus rounds as `bonusRoundsGameId` parameter in `gameConfig` section of wrapper configuration.

Example configuration:

```
{
  // rest of the configuration

  "gameConfig": {
    "bonusRoundsGameId": "extreme-dragon-el8",

    // rest of the game configuration
  }
}
```

Errors

Most errors will be handled by the `GameWrapper`, but they will also be forwarded to the game client through a rejected RGS Promise. The game client should handle any errors and special situations according to the Error Handling Guidelines.

GameWrapper states

`GameWrapper` can have one of several different states. The current state can be checked by reading the `GameWrapper.prototype.state` property.

Possible states and their meanings:

- **active** - The wrapper is active and the game can proceed normally.
- **paused** - The wrapper has issued a pause event (e.g. due to a message dialog). The game should wait for a **game-resume** event before proceeding with its next action.
- **error** - An unrecoverable error has occurred. The game won't be able to make any RGS calls. The wrapper will either reload to game or forward the player to lobby, depending on the player's choice and provider requirements.

Example

```
if (wrapper.state === 'active') {
  wrapper.rgs.refresh().then(handleRefresh).catch(handleErrors);
}
else {
  wrapper.rgs.once('game-resume', () => {
    wrapper.rgs.refresh().then(handleRefresh).catch(handleErrors);
  });
}
```

GameWrapper configuration

The GameWrapper configuration object is set during initialization and should be accessed only after wrapper initialization is finished (after `wrapper.initialized` promise is resolved).

Two methods are available for retrieving wrapper configuration: * `getConfig()` - get full wrapper configuration object, * `getGameConfig()` - get only the `gameConfig` section of wrapper config.

Return values of either one of these methods should be cached on game client side. These methods use deep object cloning to avoid config object contamination and shouldn't be called frequently.

Event handling

Some of the GameWrapper components will emit events that the game client can subscribe to. These events are currently implemented as `CustomEvents` and any data that they emit will be supplied through the `event.detail` property.

The general way of subscribing and unsubscribing to events is through `on()` and `off()` methods. A generic case would be:

```
// To subscribe
wrapper.<component>.on(eventName, callback)

// To unsubscribe
wrapper.<component>.off(eventName, callback)
```

For example here is how to subscribe to the `sound-mute` event provided by `GameWrapper.prototype.ui`:

```
wrapper.ui.on('sound-mute', function (evt) { console.log('sound should be muted') });
```

Stake

The wrapper keeps track of all available stakes. The current stake is displayed in the wrapper UI and can be updated and inspected through the methods specified below.

Stake list

The list of allowed stakes will be available as soon as the game setup response is received (after `getGame` or first `refresh` call). It can then be fetched by calling `wrapper.rgs.getStakeOptions()`.

Note that the list of allowed stakes may change during the game. These changes are triggered by the game operator based on their requirements.

When a stake list is changed, the wrapper will dispatch a `'set-stake-limits'` event. The game **must** listen to this event and update its stake list and current stake if necessary.

'set-stake-limits' event Fires when the list of allowed stakes, payline amounts, denomination values or subgames is changed.

It is dispatched with one argument of the following structure:

```
{
  stakes: number[];
  lines?: number[];
  denominations?: number[];
  subgames?: string[];
}
```

Where

- `stakes` is an array of allowed stakes,
- `lines` is an array of allowed payline amounts (added only in Slot games),
- `denominations` is an array of allowed denomination values (added only in Keno games),
- `subgames` is an array of allowed subgames (added only for multi-ID games).

Current stake

Stake selection is usually handled by the game client. Whenever a new stake is selected, the client must call `setBet(newStake)` to propagate the stake change to the wrapper.

To check the current stake, the game can use `getBet()`. This may be useful for interrupted games, history replay and stake limit changes.

Whenever the current stake changes, the wrapper will emit a *‘stake-change’* event.

`getBet()`

Get the current stake.

Returns a number.

Example

```
console.log('The current stake is: ' + wrapper.getBet());
```

`setBet(stake: number)`

Set a new stake value.

Arguments:

- `stake` - the new stake value.

Example

```
wrapper.setStake(10);
```

‘stake-change’ event

The *‘stake-change’* event fires whenever the current stake is changed. Current stake can be changed from the lobby side. Because of this, game client must always listen for *‘stake-change’* events and change selected stake if it’s different from the arrived value.

It is dispatched with one argument of the following structure:

```
{ stake: number }
```

Example

```
wrapper.on('stake-change', function (event) {  
    console.log('New stake is: ' + event.detail.stake);  
});
```

User interface

GameWrapper provides a common user interface for games. This includes a splash screen, info and error message display, game controls and configuration and user interface compliance features.

To enable the user interface features of the GameWrapper, it is necessary to set the `content` parameter in the wrapper configuration object during initialization. `content` can either be an `HTMLElement` that contains the game content or a query selector string that points to that element (e.g. `#game-content`).

It is possible to control which UI elements should be enabled and visible, either through wrapper configuration parameters or CSS. More details are available in the linked sections.

The user interface API is accessible through `GameWrapper.prototype.ui`.

Game title

Game title can be set in two ways:

1. by setting the `gameName` parameter in the wrapper configuration object.
2. by calling `wrapper.setName(name: string)`, where `name` is the desired game name.

Splash screen

Splash screen will show up automatically when the GameWrapper UI is initialized. It contains a logo display and a progress bar which can be stylized via custom CSS.

Unless **forced** to hide, the splash screen will remain visible at least until **all** the game wrapper assets (such as language and currency data) have been loaded. Even if the game has loaded and set loading progress to complete, the splash screen will wait for these wrapper assets and may remain visible for a short time.

To inform the game client the splash screen is about to close, the UI instance exposes a property `UI.prototype.ready`. Gameplay shouldn't start before this promise resolves.

ui.ready promise This is a promise that resolves when the UI is ready to show the game, i.e. when loader progress is complete and all the wrapper assets have been loaded. Any game start behaviour should be run after this promise resolves.

Example:

```
wrapper.ui.ready.then(startGame);
```

Splash screen progress bar GameWrapper's UI component provides a method for controlling the progress bar.

- `UI.prototype.setLoaderProgress(value: number)` - Set the current loading progress and update the progress bar. The value should be a percentile between 0 and 1, with 0 meaning 'nothing has been loaded' and 1 meaning 'everything loaded'.

Example:

```
function onLoadAsset(progress) {  
  wrapper.ui.setLoaderProgress(progress);  
}
```

Info bars

Unless explicitly disabled, the GameWrapper's info bars will be overlaid onto the game content. The game must take care to provide enough space, so that the bars don't cover game critical content.

There are two bars, one on the top and one on the bottom. Their heights are as follows:

- 19px on desktop,
- 30px on mobile devices.

Sound button

GameWrapper UI emits a set of events related to sound button behaviour. These are:

- **sound-mute** - Sound should be muted.
- **sound-unmute** - Sound should be unmuted.
- **sound-set-volume** - Sound volume should be updated. Will provide **one** argument containing the desired volume.

Examples:

```
wrapper.ui.on('sound-mute', function (evt) { console.log('sound should be muted') });  
wrapper.ui.on('sound-unmute', function (evt) { console.log('sound should play') });  
wrapper.ui.on('sound-set-volume', function (evt) { console.log('set volume to: ' + evt.detail);
```

Since the sound volume can also be changed from within the game client, the UI exposes a couple of methods to propagate these changes to the wrapper.

setVolume(volume: number) Set sound volume.

- *volume {number}* - volume amount in range 0-1.

Example:

```
wrapper.ui.setVolume(0.8);
```

muteSound(mute: boolean) Toggle sound mute/unmute.

Arguments

- *mute {boolean}* - flag to mute or unmute the sound. If set to **true** sound will be muted, otherwise sound will unmute.

Example:

```
wrapper.ui.muteSound(true);
```

Message screens and game pause

GameWrapper will display various messages whenever the user should be informed about the current state. This may be an error message, a regulation requirement, or a simple information like autoplay status information.

The games should pause and remain in their current state while these messages are being displayed. The game flow must not progress.

The wrapper emits a set of events to inform the game when it should pause and when it should resume. These are dispatched by the UI instance:

- **game-pause** - fires when the game should be paused (e.g. whenever a dialog pops-up)
- **game-resume** - fires when the game should resume (e.g. an informational dialog was closed)

Examples:

```
wrapper.ui.on('game-pause', function (evt) { console.log('pausing game') });  
wrapper.ui.on('game-resume', function (evt) { console.log('resuming game') });
```

Custom messages

The UI API also provides a method for showing custom informational messages. These are simple message dialogs that show a message and a single confirmation button. When the button is pressed the game resumes and there is no further action.

showMessage(message: string | MessageInterface) Show a custom informational message.

Arguments:

- **message** - a string or a **MessageInterface** object which contains the message content.

A **MessageInterface** object has the following structure (all properties are optional):

```
MessageInterface {  
  title?: number;  
  subtitle?: number;  
  text?: number;  
}
```

Example:

```
// simple message
wrapper.ui.showMessage("Enjoy the holiday special!");

// using MessageInterface
wrapper.ui.showMessage({ title: "Congratulations", text: "It's your birthday!"});
```

Win amount value and display update

GameWrapper keeps track of the current win amount. By default, the win amount label in the bottom bar will be updated when the `close` response is received. Value of win amount can be obtained by calling:

```
wrapper.getWinAmount()
```

The game can also trigger a manual label update (for example to synchronize win amount display with in-game animations). This is done by calling:

```
wrapper.ui.updateWinAmount()
```

This method will update the win amount label with the win amount for the current spin.

Turbo mode

Some games implement turbo mode which increases gameplay speed. While the wrapper doesn't implement any turbo controls, it can mediate turbo status between the game and the lobby. Because a lobby may implement turbo controls, games should update turbo status whenever it changes and should also listen to the `'set-turbo'` event.

Turbo behaviour is part of the UI API.

setTurbo(turbo: boolean) Enable or disable turbo mode.

- turbo {boolean} - true to enable turbo, false to disable it

Example:

```
wrapper.ui.setTurbo(true);
```

'set-turbo' event Fires when turbo state should be changed.

It is dispatched with one **boolean** argument.

Example:

```
wrapper.ui.on('set-turbo', function (event) {
    let turbo = event.detail;
    game.setTurbo(turbo);
});
```

Autoplay dialog

The autoplay settings dialog is part of the game wrapper UI and can be triggered from the interface. It can also be displayed manually from code, by calling:

```
wrapper.ui.showAutoplayDialog()
```

Autoplay dialog events The UI interface will dispatch a couple of events related to autoplay dialog:

- `autoplay-dialog-opened` - when autoplay dialog is opened.
- `autoplay-dialog-closed` - when autoplay dialog is closed without starting autoplay.

Autoplay duration options Autoplay dialog displays different choices for number of automatic plays (5, 10, etc.). These choices are populated from `autoplayOptions` property in the `gameConfig` section of wrapper configuration.

The list of choices is set only once during wrapper initialization and won't change during runtime.

Fullscreen

The wrapper handles fullscreen behaviour on mobile. When the game isn't in fullscreen mode, the wrapper will show an appropriate animation instructing the player on how to switch to fullscreen. It can be separately configured for android and iOS.

The `GameWrapper` object has a property `fullscreen` which provides access to the wrapper's fullscreen API. It can be used to subscribe to fullscreen events or to check the current fullscreen state.

`isActive()`

Returns `true` if fullscreen mode is active, or `false` otherwise.

On iPhones fullscreen mode is active when the address bar is minimized. On iPads fullscreen will never be active.

On Android devices fullscreen mode is active when the game is running in true fullscreen.

Example

```
console.log('Is fullscreen active? ' + wrapper.fullscreen.isActive());
```

'change' event

The Fullscreen object will fire a 'change' event whenever the fullscreen state changes.

Example

```
wrapper.fullscreen.on('change', function (evt) {  
    console.log ('Fullscreen on: ' + wrapper.fullscreen.isActive());  
});
```

URL parameters

The wrapper provides a static function for checking the provider-supplied parameters:

```
GameWrapper.getParam(key: string)
```

Argument:

- key - the parameter key

Return value

- {string} - parameter value

Currency

GameWrapper supports most of the world currencies and provides a way for games to easily display currency formatted value strings.

Currency data

The currency support in the wrapper is based on the ECMAScript Internationalization API, which provides support for most of the world currencies and their formatting rules.

To allow for finer control of the currency display, it is possible to force a certain currency to always display in a single locale. This can be done by providing a currency configuration JSON file.

To explain when this may be necessary, here is an example.

When the Swedish Kronor (SEK) is displayed with most of the world currencies the output is usually:

100 SEK

But if the locale is set to Swedish ('sv'), then the output becomes something that is more common in Sweden:

100 kr

Configuration To load a custom currency configuration, the `currencyData` parameter of the game wrapper configuration must be specified. It should be a path to the configuration JSON.

The configuration structure is as follows:

```
{
  [currency: string]: {
    code: string,
    locale: string
  }
}
```

For the SEK example above, to always force a Swedish locale resulting configuration file would look like this:

```
{
  "SEK": {
    "code": "SEK",
    "locale": "sv"
  }
}
```

Currency format

The GameWrapper instance provides a utility function for formatting currency values:

```
formatCurrency(value: number, trimFraction?: boolean, currencyCode?: string)
```

Arguments:

- **value** - the number value to be converted
- **trimFraction** - (*optional*) if true, trim the fractional part of the output for integer values
- **currencyCode** - (*optional*) ISO 4217 three letter currency code. If this value isn't specified, the currency provided by the wallet will be used.

Return value:

- **{string}** - a formatted string containing the supplied value with proper delimiters and currency symbol.

History replay

History replay is a feature which allows players and operators to replay any previously played games.

When history replay mode is active, the wrapper UI will display an indicator in the bottom bar, noting that the game is in history replay. Furthermore, the wrapper will display pop-up dialogs to inform the players when replay mode starts and when it ends.

When history replay is active, `wrapper.gameMode` will be set to *'history'*.

During replay, the game **mustn't make RGS calls that progress the game flow** and it won't be possible to do so. If these methods are called during history

replay, they will reject with the reason `historyReplay`.

Methods

The `GameWrapper` instance provides a simple API for starting and ending replays.

The wrapper will enter history replay mode automatically if there is a `ticketId` URL parameter. Otherwise, the game can start the replay mode manually by executing a `startReplay(ticketId?: string)` call.

When the game client has completed the replay, it should call `endReplay()` to notify the wrapper that the replay is over.

Method details

- `startReplay(ticketId?: string)` - Request ticket and initiate replay. If `ticketId` isn't specified, replay will start with the `ticketId` supplied as a URL parameter. This method returns a Promise of ticket data. The ticket data contains the spin and all free spins (if there are any) in the same format as regular calls. See example below.
- `endReplay()` - Stops the replay mode. Should be called when all game animations have finished and the game should show the "replay finished" pop-up.

Ticket data example For example ticket data, please check `ticket-data-example.json`.

Initiating replay from the lobby

The game lobby may request a history replay by supplying the `ticketId` URL parameter which contains a valid ticket id. The game wrapper will then set its game mode to *'history'*, but it is up to the game to start the replay.

The general process for starting a history replay in this case is:

1. Initialize `GameWrapper`.
2. When the game is loaded, check `wrapper.gameMode`. If it is *'history'* start the replay, otherwise proceed normally.
3. Start the replay with `wrapper.startReplay()` (no argument necessary). A pop up will show informing the player that a replay is about to start.
4. When the replay finishes call `wrapper.endReplay()`. A pop up will show allowing the player to return to lobby or resume playing the game.

Example

```
// 1. Initialize GameWrapper
let wrapper = new GameWrapper(config);

// 2. Check game mode when game loaded
```

```

wrapper.ui.ready.then(function () {
  if (wrapper.gameMode === 'history') {
    // 3. Start replay
    wrapper.startReplay().then(processTicket);
  }
}

// 4. End replay at the end
function finishGame() {
  if (gameMode === 'history') {
    wrapper.endReplay();
  }
  else {
    // regular gameplay
    // e.g. close()
  }
}

```

Initiating replays from the game

The game may also request replays (e.g. from an in-game history display). The process is similar to above, but this time the game needs to supply a ticket id:

1. Call `wrapper.startReplay(ticketId: string)`. `ticketId` should be a valid ticket id. A pop up will show informing the player that a replay is about to start.
2. When the replay finishes call `wrapper.endReplay()`. A pop up will show allowing the player to return to lobby or resume playing the game.

Autoplay

The wrapper comes with an autoplay manager which can handle autoplay behaviour for game clients.

The autoplay manager is accessible through `GameWrapper.prototype.autoplay`. It provides methods for starting and stopping autoplay.

Whenever the autoplay state changes, the manager will emit the appropriate event. You can subscribe to these by calling

```

wrapper.autoplay.on(eventName, callback);

```

Using autoplay

In most cases games won't need to trigger autoplay directly. They only need to display the wrapper's autoplay dialog and it will handle the rest.

The autoplay dialog is available through the UI module. To display it call:

```
wrapper.ui.showAutoplayDialog()
```

To handle autoplay, a game needs to subscribe to autoplay events and follow the flow below:

1. Switch game UI to autoplay mode on *'autoplay-started'*.
2. Trigger the first purchase.
3. Trigger another purchase whenever *'autoplay-next'* is received. This will happen after every round as long as autoplay is active.
4. When *'autoplay-stopped'* is fired, the autoplay is over. Switch back to regular game UI.

Autoplay methods

The autoplay manager provides methods for starting and stopping autoplay:

- `start(settings: AutoplaySettings)`
- `stop(reason: string)`

`start(settings: AutoplaySettings)` Start autoplay with the specified settings. The `AutoplaySettings` interface is explained below.

`stop(reason: string)` Stop autoplay with the given reason for stopping. Reason can be a free form string, e.g. *'user-stop'*.

AutoplaySettings interface

An `AutoplaySettings` object contains configuration data for an autoplay session.

Structure:

```
AutoplaySettings {  
    spins: number;  
    lossLimit: number;  
    singleWinLimit: number;  
    winLimit: number;  
    stopOnWin: boolean;  
    stopOnJackpot: boolean;  
}
```

Description:

- `spins` - Total number of spins that should be performed during autoplay.
- `lossLimit` - Total amount of money the player is allowed to lose before autoplay stops.
- `singleWinLimit` - The minimum single win that will stop autoplay.
- `winLimit` - The minimum amount of total winnings that will stop autoplay.
- `stopOnWin` - Whether autoplay should stop on any win.
- `stopOnJackpot` - Whether autoplay should stop on jackpot.

Autoplay events

- **autoplay-started** - fires when autoplay starts. Has **one** argument of type **AutoplaySettings**.
- **autoplay-next** - fires after an **RGS close()** call if there are more available spins in autoplay.
- **autoplay-stopped** - fires when autoplay is stopped for whatever reason. Provides **one** argument of type **string** containing the reason. Can be one of the following:
 - **losslimit-required** - Loss limit not set, but it is mandatory (**requireLossLimit** is set to true).
 - **losslimit-reached** - Loss limit was reached.
 - **single-winlimit-reached** - Single Win limit reached.
 - **winlimit-reached** - Win limit reached.
 - **stop-on-win** - “Stop on any win” was active and the player won something.
 - **stop-on-jackpot** - “Stop on jackpot” was active and the last spin contains a jackpot. Not in use at the moment.
 - **insufficient-funds** - Autoplay stopped due to insufficient funds.
 - **reality-check** - Autoplay stopped due to reality check.
 - **spin-error** - Autoplay stopped due to an error.
 - **spins-complete** - Autoplay has played through all assigned spins.
- **set-enabled** - fires when autoplay is enabled or disabled. For flows where autoplay can be disabled as a feature. Has **one** argument of type **boolean**.

Examples

Handling autoplay start:

```
wrapper.autoplay.on('autoplay-started', function(evt) {  
    console.log('autoplay started for ' + evt.detail.spins + ' spins');  
    wrapper.rgs.spin();  
});
```

Keeping the autoplay going:

```
wrapper.autoplay.on('autoplay-next', function(evt) {  
    console.log('autoplating next spin');  
    wrapper.rgs.spin();  
});
```

Handling autoplay stop:

```
wrapper.autoplay.on('autoplay-stopped', function(evt) {  
    console.log('autoplay stopped. reason: ' + evt.detail);  
});
```

To stop autoplay because of an in-game action:

```
wrapper.autoplay.stop('ingame-stop');
```

Handling enabling (disabling) autoplay:

```
wrapper.autoplay.on('set-enabled', function(evt) {
  if (evt.detail) {
    console.log('autoplay enabled');
  } else {
    console.log('autoplay disabled');
  }
});
```

Reality check

A reality check is a display that presents itself to the player after a set period of time and shows the time elapsed since the player's session began. It is a regulatory requirement in some jurisdictions and serves as a reminder for the player.

The reality check is usually provided by operators in one of two ways:

- through URL parameters
- through wallet messages

The wrapper currently supports only URL-based reality checks, but wallet message support will be added in the near future.

Little is required on the game side for reality checks to work. When a reality check is triggered it will dispatch a **game-pause** event. The game should then pause everything, as described in the message screens section. If the player wants to keep playing, a **game-resume** event will be dispatched when the reality check dialog is closed.

If autoplay is running when a reality check pop-up appears, it will be stopped. The autoplay manager will then fire the **autoplay-stopped** event with the stop reason set to **reality-check**.

Reality check through URL parameters

In this case, reality check is configured through URL parameters which are provided by the game lobby. The wrapper then sets up a timer based on these parameters. After the timer runs out the reality check triggers as soon as one of these happens:

- a purchase call is made (**spin** or **purchaseTicket**). The purchase will be prevented in this case.
- after an ongoing game is settled (after **close** or **settleTicket**).

To set up a reality check, the lobby must launch the game with the following URL parameters:

- **realityIntervalSecs** - how often should the reality check pop-up show. Number in seconds.

- **realityRemainingSecs** - *{optional}* how many seconds remain until the next reality check pop-up.
- **realityUrl** - *{optional}* a reality check URL if there is one.

Window messaging

A game lobby can communicate with the wrapper using the `window.postMessage()` method. The wrapper will parse these messages and forward them to the game if necessary.

The message structure is assumed to be a stringified JSON of the following format:

```
{
  name: string,
  data: any
}
```

Where

- **name** is the name of the event that is sent,
- **data** is an object with event data.

postMessage configuration

For window messaging to work, it must be configured through the **postMessage** parameter of the game wrapper configuration. The configuration specifies which window messages are available and how they map to the wrapper.

The configuration object structure:

```
allowedEvents: string[],
boolean?: {
  true: string,
  false: string,
}
maps?: {
  [key: string]: {
    name?: string,
    data?: { [key: string]: string }
  }
};
```

Where

- **allowedEvents** is a list of event types which can be parsed. All other messages will be ignored.
- **boolean** maps a string value to a boolean true or false. Used when converting string values to boolean.

- **maps** is a dictionary of maps between a wrapper's internal event and the operator provided event.

Maps The **maps** structure is somewhat complex in order to allow for flexibility with different operators.

- The **key** is the original event name of the dispatched event (whether it's sent by the wrapper or the game lobby)

Then, for each key:

- If **name** is set, the event will be renamed from **key** to **name** before processing.
 - If **key** is a wrapper event, it will be renamed to **name** before sending the message to the lobby.
 - If **key** is a lobby event, it will be parsed by the wrapper as if it were **name**.
- If **data** is set, data attributes will be mapped from the source event to the target event.

data is another dictionary where:

- The key is the name of the source data parameter
- The value is the name of the target data parameter

So, before a message is processed a data parameter will be renamed from **key** to **data[key]**.

Example Let's say an operator provides an event for changing the game volume and let's imagine it has the following structure:

```
{
  "name": "sound",
  "data": { "vol": "number" }
}
```

This event can then be mapped to the *'volume'* event supported by the GameWrapper. Since the *'volume'* event structure is:

```
{
  "name": "volume",
  "data": { "volume": "number" }
}
```

both the event name and the data parameter should be remapped.

First, since the original event is dispatched by the game lobby, its name, *'sound'* should be used as a key in the **maps** dictionary:

```
{
  "allowedEvents": ["sound"],
```

```

    "maps": {
      "sound": /* ... */
    }
  }

```

To map *‘sound’* to *‘volume’* the **name** parameter is set to *‘volume’*:

```

{
  "allowedEvents": ["sound"],
  "maps": {
    "sound": {
      "name": "volume"
    }
  }
}

```

To map the **vol** property to **volume**, a new entry is added to the **data** property:

```

{
  "allowedEvents": ["sound"],
  "maps": {
    "sound": {
      "name": "volume",
      "data": {
        "vol": "volume"
      }
    }
  }
}

```

This would then be the final configuration.

postMessage event types

The game wrapper supports the following postMessage events:

Incoming:

- *‘betStops’*
- *‘volume’*
- *‘updatePlayer’*
- *‘stopAutoSpin’*
- *‘turboMode’*

Outgoing:

- *‘gameReady’*
- *‘gameStart’*
- *‘gameEnd’*
- *‘freeSpinTrigger’*

- 'freeSpinEnd'
- 'bonusStart'
- 'bonusEnd'
- 'outOfCoins'
- 'toLobby'
- 'error'

'betStops' Sent by the operator when the list of allowed stakes, payline amounts, denomination values or subgames is changed.

Structure:

```
{
  name: 'betStops',
  data: {
    stakes: number[];
    lines?: number[];
    denominations?: number[];
    subgames?: string[];
  }
}
```

Where

- **stakes** is an array of allowed stakes,
- **lines** is an array of allowed payline amounts (added only in Slot games),
- **denominations** is an array of allowed denomination values (added only in Keno games),
- **subgames** is an array of allowed subgames (added only for multi-ID games).

Triggers *'set-stake-limits'*.

'volume' Sent by the operator to change game volume.

Structure:

```
{
  name: 'volume',
  data: {
    volume: number
  }
}
```

Where

- **volume** is the sound volume in range 0-1.

'updatePlayer' Sent by the operator when the player's balance has changed.

Structure:

```
{
  name: 'updatePlayer',
  data: {}
}
```

‘stopAutoSpin’ Sent by the operator to stop autoplay.

Structure:

```
{
  name: 'stopAutoSpin',
  data: {}
}
```

‘turboMode’ Sent by the operator to switch turbo mode.

Structure:

```
{
  name: 'turboMode',
  data: {
    mode: string
  }
}
```

Where

- mode is a string that maps to a boolean - true to enable turbo, false to disable it.

‘gameReady’ Sent by the wrapper when the game is ready.

Structure:

```
{
  name: 'gameReady',
  data: {
    isReady: boolean
  }
}
```

Where

- isReady is the ready state of the game

‘gameStart’ Sent by the wrapper when a ticket is purchased (spin or purchaseTicket).

Structure:

```
{
  name: 'gameStart',
  data: {
    amountBet: number
  }
}
```

Where

- `amountBet` is the total stake amount for the current game

‘gameEnd’ Sent by the wrapper when a ticket is settled (`close` or `settleTicket`).

Structure:

```
{
  name: 'gameEnd',
  data: {
    amountWon: number
  }
}
```

Where

- `amountWon` is the total amount of winnings at the end of the game.

‘freeSpinTrigger’ Sent by the wrapper when the free game is awarded.

Structure:

```
{
  name: 'freeSpinTrigger',
  data: {
    freeSpinCount: number
  }
}
```

Where

- `freeSpinCount` is the amount of won free spins.

‘freeSpinEnd’ Sent by the wrapper when the last one free game spin is done.

Structure:

```
{
  name: 'freeSpinEnd'
}
```


‘bonusStart’ Sent by the wrapper when bonus game starts. Has to be manually triggered by calling `wrapper.messaging.sendBonusStart()`.

Structure:

```
{
  name: 'bonusStart'
}
```

‘bonusEnd’ Sent by the wrapper when bonus game ends. Has to be manually triggered by calling `wrapper.messaging.sendBonusEnd()`.

Structure:

```
{
  name: 'bonusEnd',
  data: {
    bonusAmountWon: number,
    totalBonusAmountWon: number
  }
}
```

Where

- `bonusAmountWon` is the amount won during the last bonus round.
- `totalBonusAmountWon` is the total amount of all bonus wins (if the game accumulates them).

‘outOfCoins’ Sent by the wrapper if a purchase is attempted with insufficient funds.

Structure:

```
{
  name: 'outOfCoins',
  data: {
    amountBet: number,
  }
}
```

Where

- `amountBet` is the stake value of the attempted purchase.

‘toLobby’ Sent by the wrapper when the button To lobby is clicked.

Structure:

```
{
  name: 'toLobby',
  data: {
```

```

    detail: string | null,
  }
}

```

Where

- `detail` might be a reason why `To lobby` is activated (e.g `sessionTimeout`) and it's related to the RGS's error. If the value is `null`, that means there isn't an error and event is triggered from the ordinary flow (e.g from Menu's `To Lobby` button)

'error' Sent by the wrapper if an error occurs.

Structure:

```

{
  name: 'error',
  data: {
    error: any,
  }
}

```

Where

- `error` is an arbitrary error object.

Translations

The wrapper has support for multiple languages and customized translations can be easily added.

The language data should be stored in the following path: `gamewrapper/lang/<language>.json`, where `<language>` is the desired language code. If the wrapper fails to find a language file, it will fall back to a default set of English language strings.

Appendix: Game client configuration

GameWrapper configuration is required to contain a `gameConfig` section with some obligatory parameters depending on the game type.

This section should also be used for storing any additional configuration data that the game client requires.

Game configuration

`gameConfig` section is set during initialization (together with wrapper configuration object) and should be accessed only after wrapper initialization is finished (after `wrapper.initialized` promise is resolved).

`wrapper.getGameConfig()` should be called to receive a copy of `gameConfig` section. Return value of this method should be cached on game client side. This method uses deep object cloning to avoid config object contamination and shouldn't be called frequently.

In case of upgrading from a version of `GameWrapper` that expected to store these values in `clientConfiguration` section of RGS configuration file: move all contents from that section into `gameConfig` section of wrapper configuration.

Slots game client configuration

`gameConfig` for slot games has following mandatory properties:

```
{
  stakeValues: number[], // available stake values
  defaultStake: number, // default stake (recommended, but not mandatory)
  defaultLineCount: number, // default line count
  autoplayOptions: number[] // autoplay duration (mandatory if autoplay is needed)
}
```

Keno game client configuration

`gameConfig` for keno games has following mandatory properties:

```
{
  stakeValues: number[], // available stake values
  defaultStake: number, // default stake (recommended, but not mandatory)
  denomValues: number[], // denomination values
  autoplayOptions: number[] // autoplay duration (mandatory if autoplay is needed)
}
```

Cards game client configuration

`gameConfig` for keno games has following mandatory properties:

```
{
  stakeValues: number[], // available stake values
  defaultStake: number, // default stake (recommended, but not mandatory)
  autoplayOptions: number[] // autoplay duration (mandatory if autoplay is needed)
}
```

Fixed-odds game client configuration

`gameConfig` for fixed-odds games has following mandatory properties:

```
{
  initialStakeIndex: number, // default stake index (recommended, but not mandatory)
  autoplayOptions: number[] // autoplay duration (mandatory if autoplay is needed)
}
```

CDN Support

Game wrapper offers dynamic CDN URLs support:

1. It's enabled by default.
2. If an URL is given in absolute form, it wouldn't be updated to point to CDN. Just remove unnecessary params from config (or use relative URLs in them).
3. If needed, this functionality can be disabled by setting `isCdnSupportEnabled` to false in wrapper's config file.

Language configuration

Configuration stored in `language-config.json` lets you remap a URL language code to a different code, change path where language files are stored and .

Language config example:

```
{
  "langPath": "lang/",
  "langMap": { "se": "sv" },
  "enableLangAttribute" : false
}
```

- `langPath {string}` - (*optional*) relative or absolute path to the directory which contains the GameWrapper language files. The default path is `gamewrapper/lang`.
- `langMap {dictionary}` - (*optional*) map one language code to another code.
- `enableLangAttribute {boolean}` - (*optional*) allows adding lang attribute to the main html document, enabled by default. . # Jurisdictions Game wrapper's game config could be overridden depending of jurisdiction when game is launched.

The jurisdiction data should be stored in the following path: `gamewrapper/jurisdiction/<jurisdiction>.json` where `<jurisdiction>` is the desired jurisdiction code that must match the parameter received during game launch.

Every jurisdiction config file can have `gameConfig` or `wrapperConfig` part.

Game config

`gameConfig` part rewrites the specific options of the game configuration, what is specified in this part will directly overwrite the game config part of the game.

Jurisdiction game config example:

```

{
  "gameConfig" : {
    "maxAutoplayOption" : 50, // set autoplay limit
    "hasSpinStop" : true,
    "hasAutoPlay" : false
  }
  // rest of the configuration
}

```

Wrapper config

`wrapperConfig` is used for specific options of the game configuration, may contain `ui` and `additional` part.

`ui` settings are used to overwrite bottombar and topbar game configs, `additional` settings are used for special wrapper options.

Jurisdiction wrapper config example:

```

{
  "wrapperConfig": {
    "ui" : {
      "bottomBar" : {
        "elapsedTime" : true,
        "lblNetAmount" : false
      },
      "topBar" : {
        "elapsedTime" : false
      }
    },
    "additional" : {
      "gameName" : true,
      "clock" : false,
      "date" : true
    }
  }
  // rest of the configuration
}

```

Where - `gameName` {boolean} - (optional) Enable/disable name of game. - `clock` {boolean} - (optional) Enable/disable clock. - `date` {boolean} - (optional) Enable/disable date.

`showJurisdiction()` Displaying jurisdiction config file.

`wrapper.showJurisdiction()`

UKGC

getNetAmount() The net position is defined as the total of all winnings minus the sum of all losses since the start of the session.

```
wrapper.getNetAmount()
```

getElapsedTime() The elapsed time should be displayed for the duration of the gaming session. Elapsed time should be displayed in seconds, minutes, and hours.

```
wrapper.getElapsedTime() // return an object {hours, minutes, seconds}
```

Cache busting

In game wrapper resources can be fetched with query that have version with value of current timestamp. Using versioned resources in this way ensures that resources are not cached in the browser.

```
wrapper.getVersionedResourceUrl()
```